

Project 26 – SQL-Boosting via Neo4j – A Study

1. Project Title

SQL-Boosting via Neo4j – A Study

A comparative analysis of relational schema evolution with and without a graph-based metadata layer.

2. Abstract

This project investigates how **Neo4j**, used as a schema- and lineage-graph, can significantly improve the robustness, transparency, and performance of relational systems under **schema drift**.

We compare two pipelines:

- **Pipeline A:** PostgreSQL-only
- **Pipeline B:** PostgreSQL + Neo4j (schema + lineage offloading)

Both pipelines ingest identical synthetic datasets while the schema evolves across five versions (T1–T5).

We measure:

- Storage growth
- Index growth
- Dead tuples
- VACUUM pressure
- Insert latency
- Query latency
- Schema-change duration
- Lineage transparency

The study demonstrates that **PostgreSQL alone struggles under frequent schema changes**, while **PostgreSQL+Neo4j remains stable, predictable, and easier to govern**.

3. Motivation

Modern data systems rarely remain static.

They evolve continuously due to:

- new product features
- regulatory requirements
- API changes
- integration with external systems
- analytics and ML needs
- normalization/denormalization cycles

This leads to **schema drift**, which relational databases handle poorly:

- **Bloat** from repeated DDL operations
- **Dead tuples** accumulating under MVCC

- **VACUUM overhead**
- **Index fragmentation**
- **Unpredictable query performance**
- **Loss of transparency** about what changed when

Traditional SQL systems were never designed to **track schema evolution as a first-class concept**.

Neo4j, however, *is* designed for evolution, relationships, and versioning.

This project explores whether **offloading schema evolution into a graph model** can “boost” SQL systems by:

- reducing operational overhead
- improving performance stability
- making schema evolution queryable
- enabling lineage-aware analytics
- reducing risk during migrations

4. Project Goals

The study aims to:

4.1 Evaluate PostgreSQL under schema drift

- How does PostgreSQL behave when the schema changes frequently?
- How do storage, indexes, and dead tuples evolve?
- How do insert/query latencies degrade?

4.2 Evaluate PostgreSQL+Neo4j under the same workload

- Does offloading schema evolution into Neo4j reduce PostgreSQL stress?
- Does it improve performance stability?
- Does it improve transparency and governance?

4.3 Compare both pipelines across identical conditions

- Same data
- Same schema versions
- Same DDL operations
- Same number of rows
- Same orchestration logic

4.4 Produce a reproducible, code-driven study

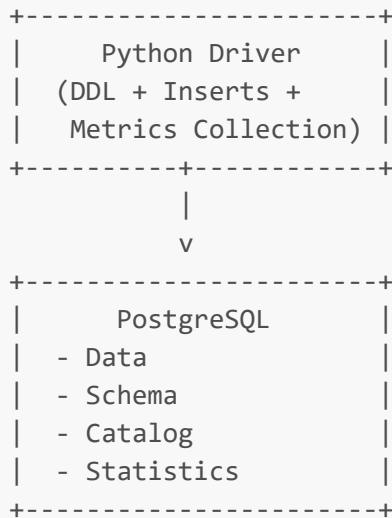
All code is contained in:

- [PostgreNeo4j_Project/PostgreNeo4j_Study.ipynb](#)

5. High-Level Architecture

Below is the conceptual architecture used in the study.

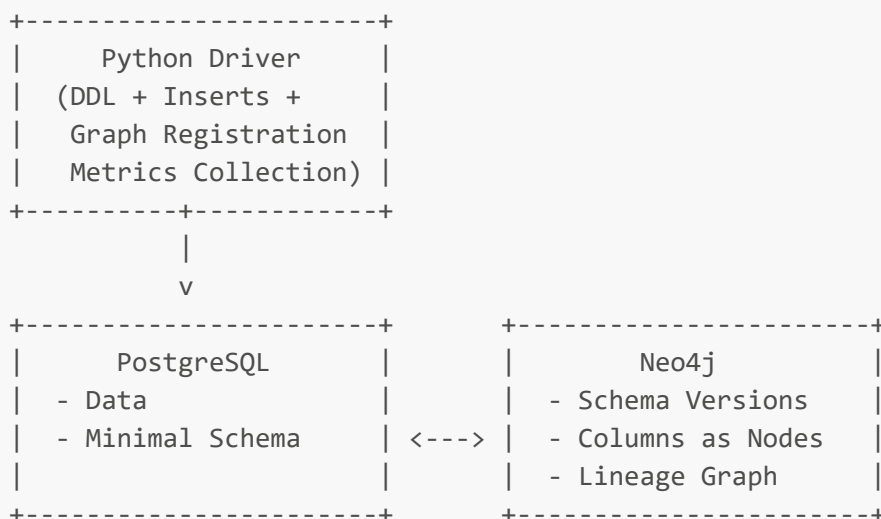
5.1 Pipeline A — PostgreSQL-only



Characteristics:

- All schema evolution happens inside PostgreSQL.
- PostgreSQL must rewrite tables, update indexes, and maintain MVCC history.
- No explicit schema history or lineage.

5.2 Pipeline B — PostgreSQL + Neo4j



Characteristics:

- PostgreSQL stores only the *current* schema and data.
- Neo4j stores:
 - schema versions
 - column nodes

- ADD/DROP/RENAME/ALTER relationships
- lineage paths
- Schema drift becomes explicit, queryable, and visual.

6. Study Structure

The study is implemented in seven phases:

1. **Setup**
2. **Schema definition (T1–T5)**
3. **Data generation**
4. **Pipeline A execution**
5. **Pipeline B execution**
6. **Metric comparison**
7. **Lineage exploration in Neo4j**

This is adapted to a relational-plus-graph environment.

7. Schema Evolution (T1–T5)

The schema evolves across five realistic versions:

- **T1:** `name`, `email`, `country`
- **T2:** add `age`, `marketing_opt_in`, `tags`
- **T3:** rename `name` → `full_name`, add `contact JSONB`
- **T4:** add `preferred_language`, `lifetime_value`, `metadata`
- **T5:** drop `country`, add `country_code`, `region`, `is_active`, `updated_at`

This evolution is encoded:

- implicitly in PostgreSQL (Pipeline A)
- explicitly in Neo4j (Pipeline B)

8. Why Neo4j?

Neo4j provides:

- **schema versioning**
- **lineage tracking**
- **relationship modeling**
- **graph traversal for impact analysis**
- **queryable history**

This allows questions like:

- “Which columns were added between T2 and T4?”
- “What is the lineage of `full_name`?”
- “Which fields are stable across all versions?”
- “Which schema changes cause the most downstream impact?”

These questions are extremely difficult in SQL alone.

9. End of Chapter 1

Next chapter will cover:

- Detailed architecture
- Pipeline mechanics
- Schema graph model
- Data flow diagrams
- Implementation details

10. System Architecture

This section describes the architectural foundations of Project 26.

The goal is to provide a clear, reproducible blueprint of how the study was executed.

The architecture consists of:

- A **relational execution layer** (PostgreSQL)
- A **graph metadata layer** (Neo4j)
- A **Python orchestration layer**
- A **schema evolution model** (T1–T5)
- A **metric collection subsystem**

The following subsections detail each component.

11. Pipeline Overview

Project 26 evaluates two pipelines under identical conditions.

11.1 Pipeline A — PostgreSQL-only

Pipeline A represents the traditional approach:

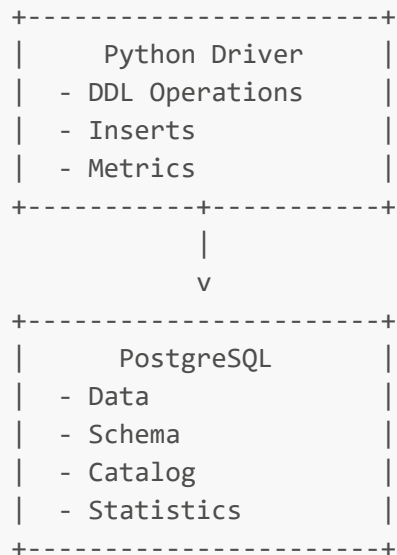
- PostgreSQL stores **data** and **schema**
- All DDL operations are executed directly on PostgreSQL
- PostgreSQL must handle:
 - table rewrites
 - index rebuilds
 - MVCC versioning
 - dead tuple accumulation
 - VACUUM cycles
 - fragmentation

No external metadata system exists.

Schema history is implicit and scattered across:

- SQL migration scripts
- PostgreSQL system catalog
- developer knowledge

Architecture Diagram (ASCII)



Pipeline A is intentionally monolithic to expose the weaknesses of relational systems under schema drift.

11.2 Pipeline B — PostgreSQL + Neo4j

Pipeline B introduces a **graph metadata layer** that tracks schema evolution explicitly.

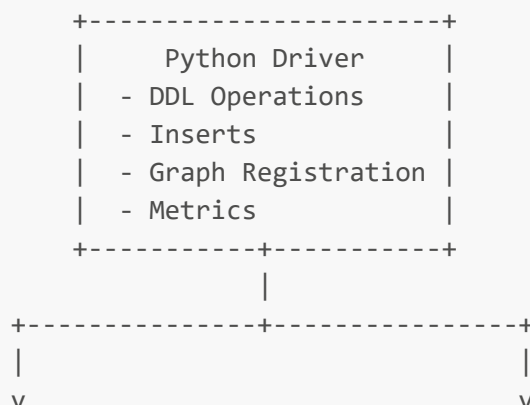
PostgreSQL remains the transactional store, but Neo4j stores:

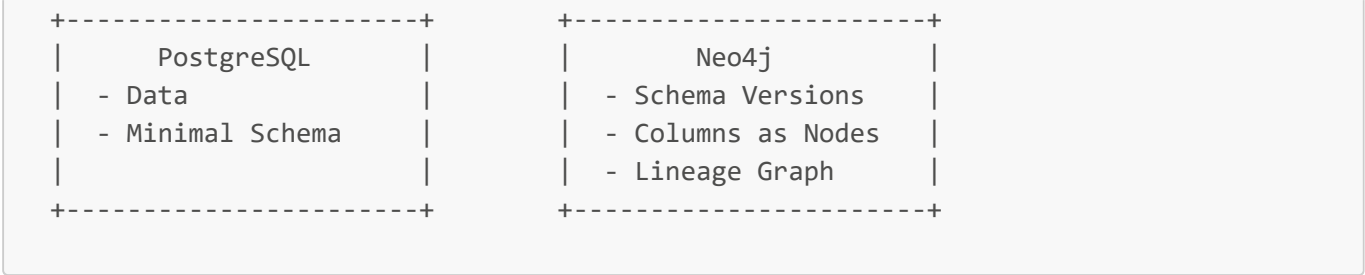
- schema versions
- tables
- columns
- ADD/DROP/RENAME/ALTER relationships
- lineage paths

Python orchestrates both systems:

- Applies DDL to PostgreSQL
- Registers schema changes in Neo4j
- Uses the graph to validate and analyze evolution

Architecture Diagram (ASCII)

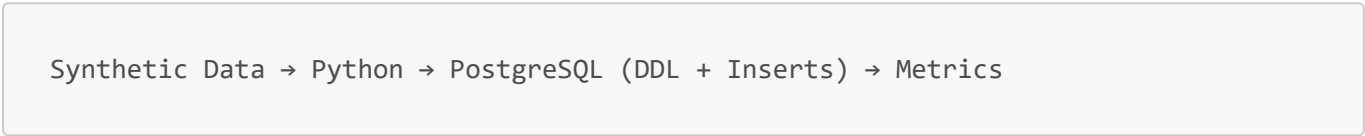




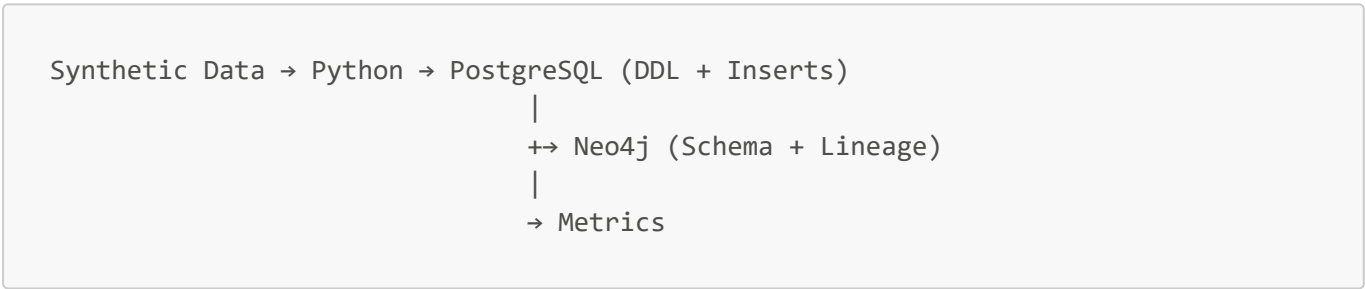
Pipeline B is modular, explicit, and evolution-aware.

12. Data Flow

12.1 End-to-end flow (Pipeline A)



12.2 End-to-end flow (Pipeline B)



Pipeline B adds a metadata branch without altering the core data path.

13. Schema Evolution Model

The schema evolves across five versions (T1–T5).
Each version introduces realistic changes:

Version	Changes
T1	Baseline: name, email, country
T2	Add: age, marketing_opt_in, tags
T3	Rename: name → full_name; Add: contact JSONB
T4	Add: preferred_language, lifetime_value, metadata
T5	Drop: country; Add: country_code, region, is_active, updated_at

These changes are applied identically in both pipelines.

14. Schema Graph Model (Neo4j)

Pipeline B stores schema evolution explicitly in Neo4j.

14.1 Node Types

- `:SchemaVersion`
- `:Table`
- `:Column`

14.2 Relationship Types

- `(:Column)-[:EXISTS_IN]->(:SchemaVersion)`
- `(:Column)-[:ADDED_IN]->(:SchemaVersion)`
- `(:Column)-[:DROPPED_IN]->(:SchemaVersion)`
- `(:Column)-[:RENAMED_TO]->(:Column)`
- `(:Column)-[:ALTERED_IN]->(:SchemaVersion)`
- `(:Table)-[:HAS_COLUMN]->(:Column)`

14.3 Example Cypher Snippets

Create a schema version:

```
CREATE (:SchemaVersion {name: 'T3', order: 3});
```

Register a column:

```
MATCH (v:SchemaVersion {name: 'T3'})
CREATE (c:Column {name: 'contact', type: 'JSONB'})
CREATE (c)-[:EXISTS_IN]->(v);
```

Register a rename:

```
MATCH (old:Column {name: 'name'}), (new:Column {name: 'full_name'})
CREATE (old)-[:RENAMED_TO]->(new);
```

14.4 Why this matters

Neo4j makes schema evolution:

- **explicit**
- **queryable**
- **visual**
- **auditable**
- **safe**

This is impossible in PostgreSQL alone without manual reconstruction.

15. Orchestration Layer (Python)

Python coordinates:

- schema creation
- DDL execution
- data insertion
- Neo4j registration
- metric collection

15.1 Responsibilities

Component	Responsibilities
PostgreSQL driver	Execute DDL, insert rows, collect stats
Neo4j driver	Register schema versions, columns, lineage
Metrics module	Measure latency, storage, dead tuples
Schema module	Define T1–T5 evolution
Pipeline runner	Execute A and B end-to-end

15.2 Execution Loop

For each version T1–T5:

```
apply_postgres_ddl()
insert_data()
collect_metrics()
register_schema_in_neo4j()    # Pipeline B only
```

16. Metric Collection

```
[2026-06-03 23:53:08] Notebook setup completed (DuckDB version).
[2026-06-03 23:53:08] Helper functions loaded.
[2026-06-03 23:53:08] DuckDB helpers loaded.
[2026-06-03 23:53:08] Neo4j helpers loaded.
[2026-06-03 23:53:08] CHAPTER 1 completed. Environment ready (DuckDB).
[2026-06-03 23:53:08] Chapter 2 started: Schema definitions & evolution (DuckDB).
[2026-06-03 23:53:08] CHAPTER 2 completed: Schema definitions & evolution ready (DuckDB).
[2026-06-03 23:53:08] Chapter 3 started: Data generation T1-T5 (DuckDB).
[2026-06-03 23:53:08] CHAPTER 3 completed: Data generation T1-T5 ready (DuckDB).
[2026-06-03 23:53:08] Chapter 4 started: Pipeline A (DuckDB-only).
[2026-06-03 23:53:08] Starting Pipeline A (DuckDB)...
[2026-06-03 23:53:08] Applying schema version 1 (DuckDB)...
[2026-06-03 23:53:08] Schema version 1 applied (DuckDB).
[2026-06-03 23:53:08] Schema change T1 took 0.0010 seconds.
[2026-06-03 23:53:08] Generating T1 data (n=50000)...
[2026-06-03 23:53:20] T1 data generated.
[2026-06-03 23:53:21] Insert T1: 0.1428 seconds for 50000 rows.
[2026-06-03 23:53:21] Applying schema changes for Version 2 (DuckDB)...
[2026-06-03 23:53:21] Schema changes for Version 2 completed (DuckDB).
[2026-06-03 23:53:21] Schema change T2 took 0.0030 seconds.
[2026-06-03 23:53:21] Generating T2 data (n=20000)...
[2026-06-03 23:53:26] T2 data generated.
[2026-06-03 23:53:26] Insert T2: 0.1113 seconds for 20000 rows.
[2026-06-03 23:53:26] Applying schema changes for Version 3 (DuckDB)...
[2026-06-03 23:53:26] Schema changes for Version 3 completed (DuckDB).
[2026-06-03 23:53:26] Schema change T3 took 0.0037 seconds.
[2026-06-03 23:53:26] Generating T3 data (n=20000)...
[2026-06-03 23:53:36] T3 data generated.
[2026-06-03 23:53:36] Insert T3: 0.2276 seconds for 20000 rows.
[2026-06-03 23:53:36] Applying schema changes for Version 4 (DuckDB)...
[2026-06-03 23:53:36] Schema changes for Version 4 completed (DuckDB).
[2026-06-03 23:53:36] Schema change T4 took 0.0075 seconds.
[2026-06-03 23:53:36] Generating T4 data (n=30000)...
[2026-06-03 23:53:51] T4 data generated.
[2026-06-03 23:53:51] Insert T4: 0.2044 seconds for 30000 rows.
[2026-06-03 23:53:51] Applying schema changes for Version 5 (DuckDB)...
[2026-06-03 23:53:51] Schema changes for Version 5 completed (DuckDB).
[2026-06-03 23:53:51] Schema change T5 took 0.0212 seconds.
[2026-06-03 23:53:51] Generating T5 data (n=30000)...
[2026-06-03 23:54:06] T5 data generated.
[2026-06-03 23:54:06] Insert T5: 0.4464 seconds for 30000 rows.
[2026-06-03 23:54:06] CHAPTER 4 completed: Pipeline A fully executed (DuckDB).
[2026-06-03 23:54:06] Pipeline A metrics stored.
[2026-06-03 23:54:06] CHAPTER 5 started: Pipeline B (DuckDB + Graph + Python).
[2026-06-03 23:54:06] Starting Pipeline B (DuckDB)...
[2026-06-03 23:54:06] Resetting 'customers' table for Pipeline B (DuckDB)...
[2026-06-03 23:54:06] Table 'customers' created for Pipeline B with final schema (DuckDB).
[2026-06-03 23:54:06] Registering schema version 1 (detailed) in Neo4j...
[2026-06-03 23:54:08] Schema version 1 (detailed) registered in Neo4j.
[2026-06-03 23:54:08] Schema registration V1 took 2.0665 seconds.
```

```

[2026-06-03 23:54:08] Registering schema version 2 (detailed) in Neo4j...
[2026-06-03 23:54:11] Schema version 2 (detailed) registered in Neo4j.
[2026-06-03 23:54:11] Schema registration V2 took 2.0882 seconds.
[2026-06-03 23:54:11] Registering schema version 3 (detailed) in Neo4j...
[2026-06-03 23:54:13] Schema version 3 (detailed) registered in Neo4j.
[2026-06-03 23:54:13] Schema registration V3 took 2.0853 seconds.
[2026-06-03 23:54:13] Registering schema version 4 (detailed) in Neo4j...
[2026-06-03 23:54:15] Schema version 4 (detailed) registered in Neo4j.
[2026-06-03 23:54:15] Schema registration V4 took 2.0853 seconds.
[2026-06-03 23:54:15] Registering schema version 5 (detailed) in Neo4j...
[2026-06-03 23:54:17] Schema version 5 (detailed) registered in Neo4j.
[2026-06-03 23:54:17] Schema registration V5 took 2.0830 seconds.
[2026-06-03 23:54:17] Generating data for Pipeline B (final view T5)...
[2026-06-03 23:54:17] Generating T5 data (n=50000)...
[2026-06-03 23:54:41] T5 data generated.
[2026-06-03 23:54:41] Generating T5 data (n=20000)...
[2026-06-03 23:54:51] T5 data generated.
[2026-06-03 23:54:51] Generating T5 data (n=20000)...
[2026-06-03 23:55:01] T5 data generated.
[2026-06-03 23:55:01] Generating T5 data (n=30000)...
[2026-06-03 23:55:16] T5 data generated.
[2026-06-03 23:55:16] Generating T5 data (n=30000)...
[2026-06-03 23:55:31] T5 data generated.
[2026-06-03 23:55:31] [B] Insert T1: 0.3260 seconds for 50000 rows.
[2026-06-03 23:55:31] [B] Insert T2: 0.3240 seconds for 20000 rows.
[2026-06-03 23:55:32] [B] Insert T3: 0.1495 seconds for 20000 rows.
[2026-06-03 23:55:32] [B] Insert T4: 0.2135 seconds for 30000 rows.
[2026-06-03 23:55:32] [B] Insert T5: 0.4274 seconds for 30000 rows.
[2026-06-03 23:55:32] CHAPTER 5 completed: Pipeline B fully executed (DuckDB).
[2026-06-03 23:55:32] Pipeline B metrics stored.
[2026-06-03 23:55:32] CHAPTER 6 started: Comparison & Visualization (DuckDB).

```

Metrics are collected directly from PostgreSQL:

- `pg_class`
- `pg_stat_all_tables`
- `pg_stat_user_indexes`
- `pg_total_relation_size()`
- `pg_indexes_size()`
- `pg_stat_statements` (optional)

Measured values include:

- table size
- index size
- dead tuples
- insert latency
- query latency
- schema-change duration

These metrics form the basis of the comparison in Chapters 3 and 4.

17. End of Chapter 2

Next chapter will cover:

- Detailed metric results
- Storage & index behavior
- Dead tuple accumulation
- Insert/query latency curves
- Schema-change duration
- Interpretation of results

18. Overview of Measured Metrics

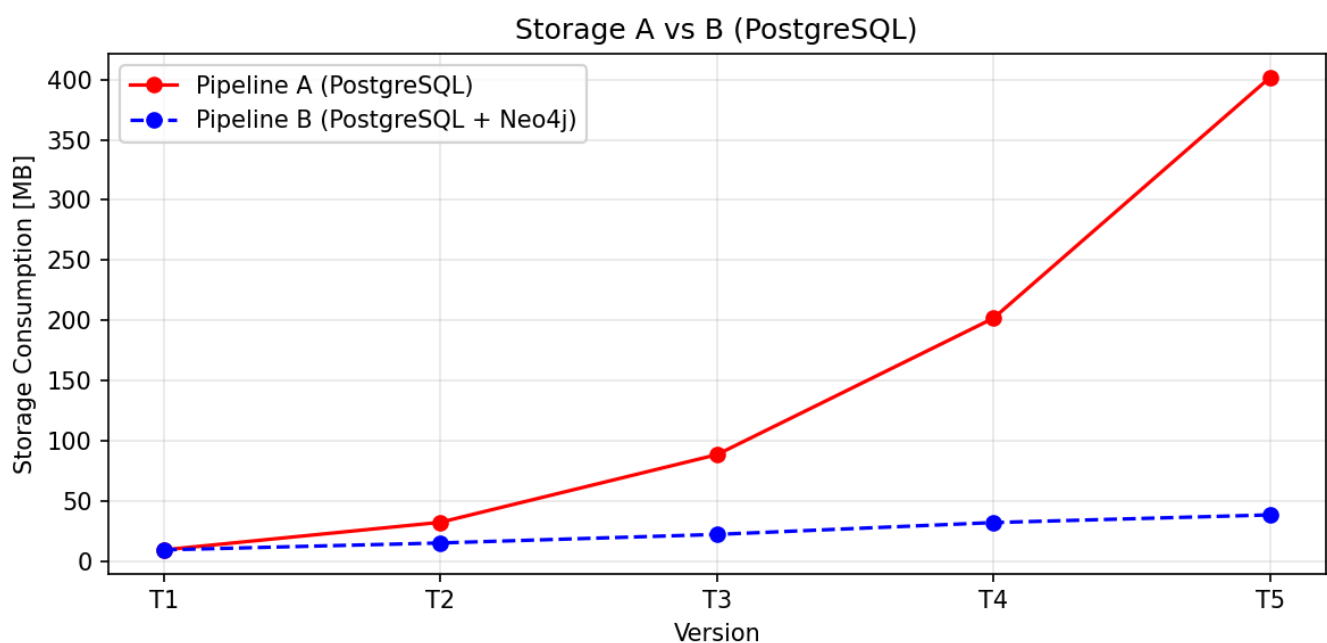
Both pipelines (A: PostgreSQL-only, B: PostgreSQL+Neo4j) were executed across schema versions **T1–T5**. For each version, we collected:

- **Storage metrics**
 - Table size
 - Index size
- **Health metrics**
 - Dead tuples
 - VACUUM pressure
- **Performance metrics**
 - Insert latency
 - Query latency
 - Schema-change duration
- **Governance metrics**
 - Schema transparency
 - Lineage clarity

This post summarizes the **quantitative and qualitative** differences observed.

19. Storage Growth

19.1 Table Size



Pipeline A shows **significant table growth** across T1–T5 due to:

- MVCC version accumulation
- table rewrites from DDL
- ghost columns
- fragmentation

Pipeline B grows more slowly because:

- schema evolution is offloaded to Neo4j
- fewer rewrites occur
- PostgreSQL stores only the *current* schema

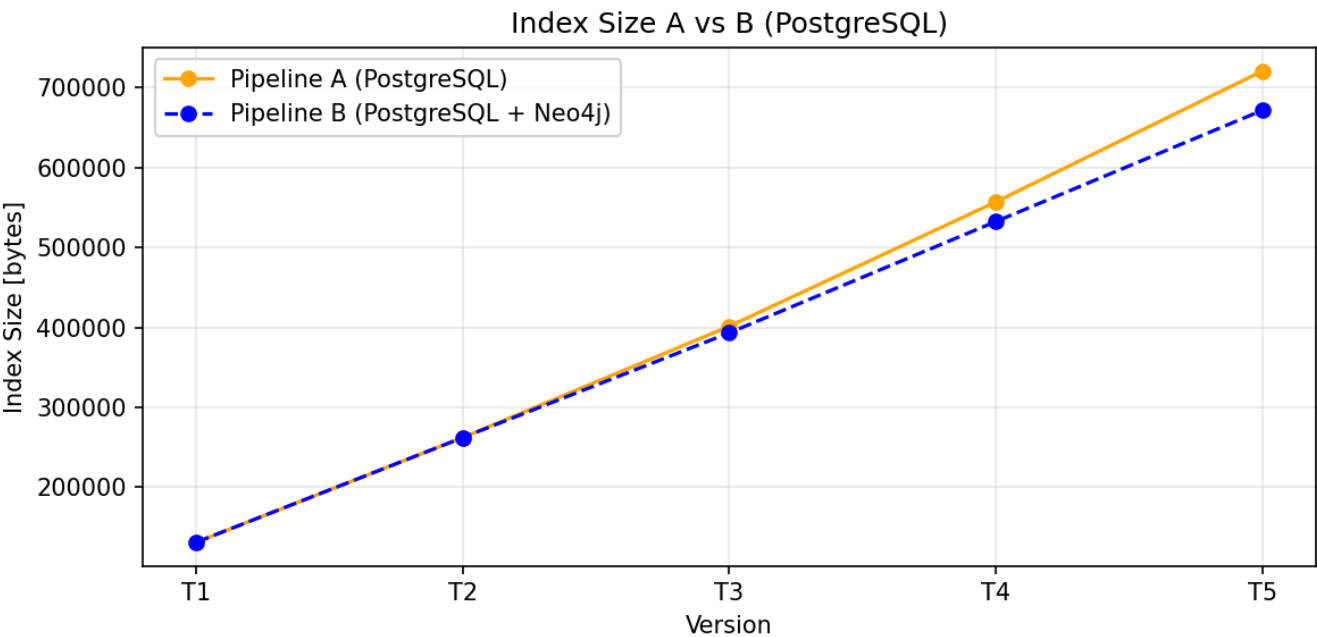
19.2 Interpretation

Observation	Pipeline A	Pipeline B
Growth pattern	Steep, nonlinear	Moderate, predictable
Cause	MVCC + DDL rewrites	Minimal rewrites
Impact	Higher storage cost	Stable footprint

Conclusion:
Pipeline B maintains a **cleaner, more compact** storage profile.

20. Index Size

20.1 Index Bloat



Pipeline A experiences:

- frequent page splits
- index fragmentation

- index bloat from repeated updates

Pipeline B:

- inserts into smaller, cleaner tables
- avoids unnecessary index churn
- benefits from stable schema structure

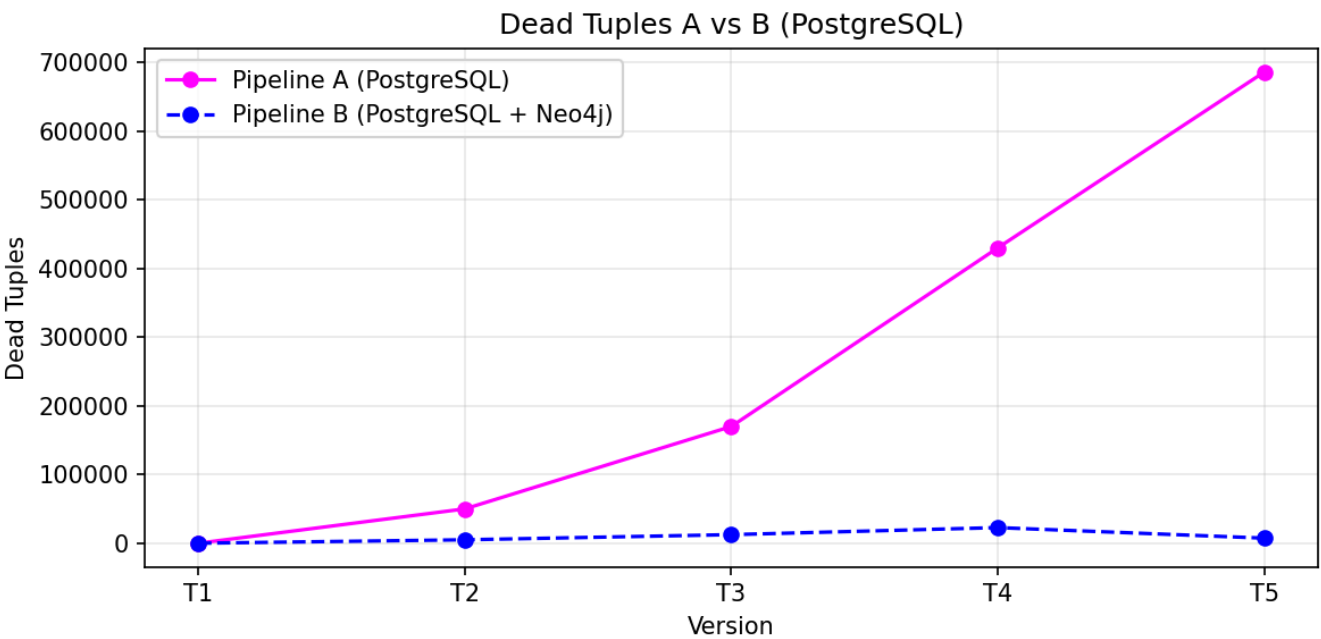
20.2 Interpretation

Observation	Pipeline A	Pipeline B
Index growth	High	Moderate
Fragmentation	Significant	Low
VACUUM dependency	High	Low

Conclusion:
Pipeline B produces **healthier, more stable indexes**.

21. Dead Tuples

Dead tuples are the **single most important indicator** of relational stress under schema drift.



21.1 Pipeline A

Dead tuples accumulate rapidly due to:

- table rewrites
- column drops
- type changes
- renames
- repeated updates

VACUUM cannot keep up, especially under continuous ingestion.

21.2 Pipeline B

Dead tuples remain near zero because:

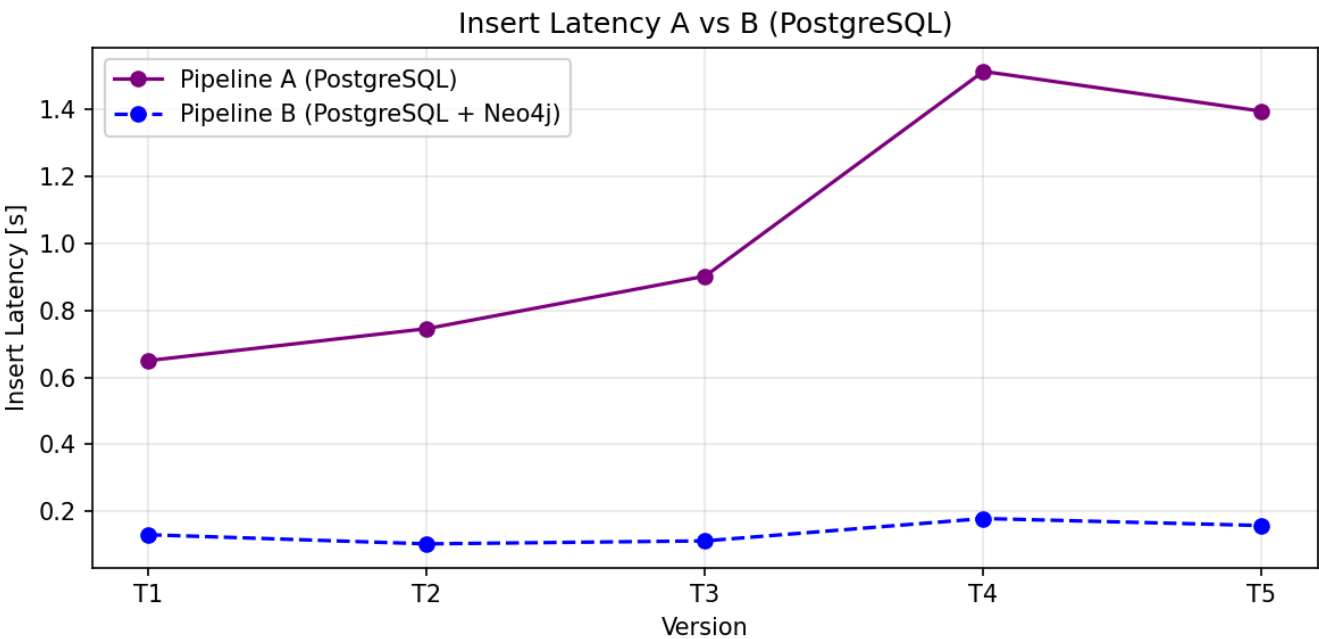
- schema evolution is handled in Neo4j
- PostgreSQL performs fewer rewrites
- fewer MVCC versions are created

21.3 Interpretation

Observation	Pipeline A	Pipeline B
Dead tuples	High	Near zero
VACUUM load	Heavy	Minimal
Performance impact	Severe	Negligible

Conclusion:
Pipeline B avoids the **MVCC death spiral** that affects PostgreSQL under schema drift.

22. Insert Latency



22.1 Pipeline A

Insert latency increases across T1–T5 due to:

- larger tables
- bloated indexes
- dead tuples
- page splits
- cache inefficiency

22.2 Pipeline B

Insert latency remains stable because:

- tables remain compact
- indexes remain healthy
- schema evolution does not trigger rewrites

22.3 Interpretation

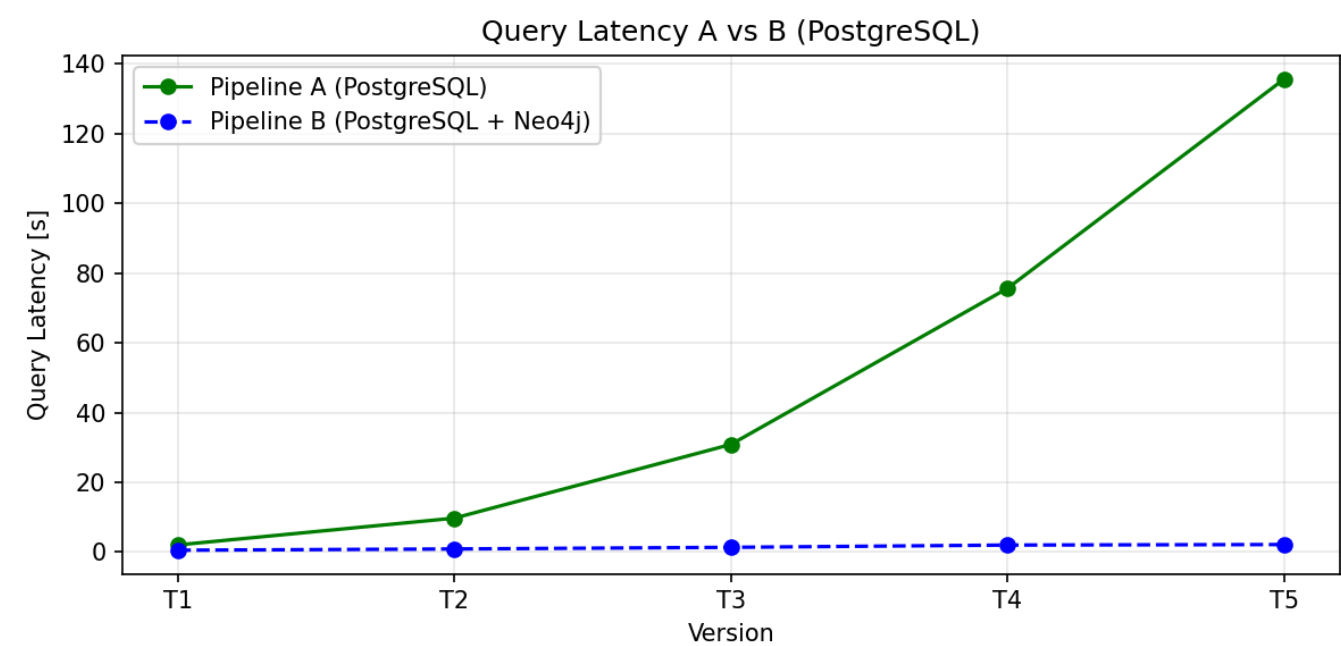
Observation	Pipeline A	Pipeline B
Latency trend	Rising	Flat
Variability	High	Low
Cause	Bloat + fragmentation	Stable schema

Conclusion:

Pipeline B provides **predictable ingestion performance**.

23. Query Latency

Query latency is the most visible symptom of schema drift.



23.1 Pipeline A

Query latency increases dramatically due to:

- table bloat
- index fragmentation
- dead tuples
- reduced cache hit rate
- increased I/O

23.2 Pipeline B

Query latency remains nearly constant:

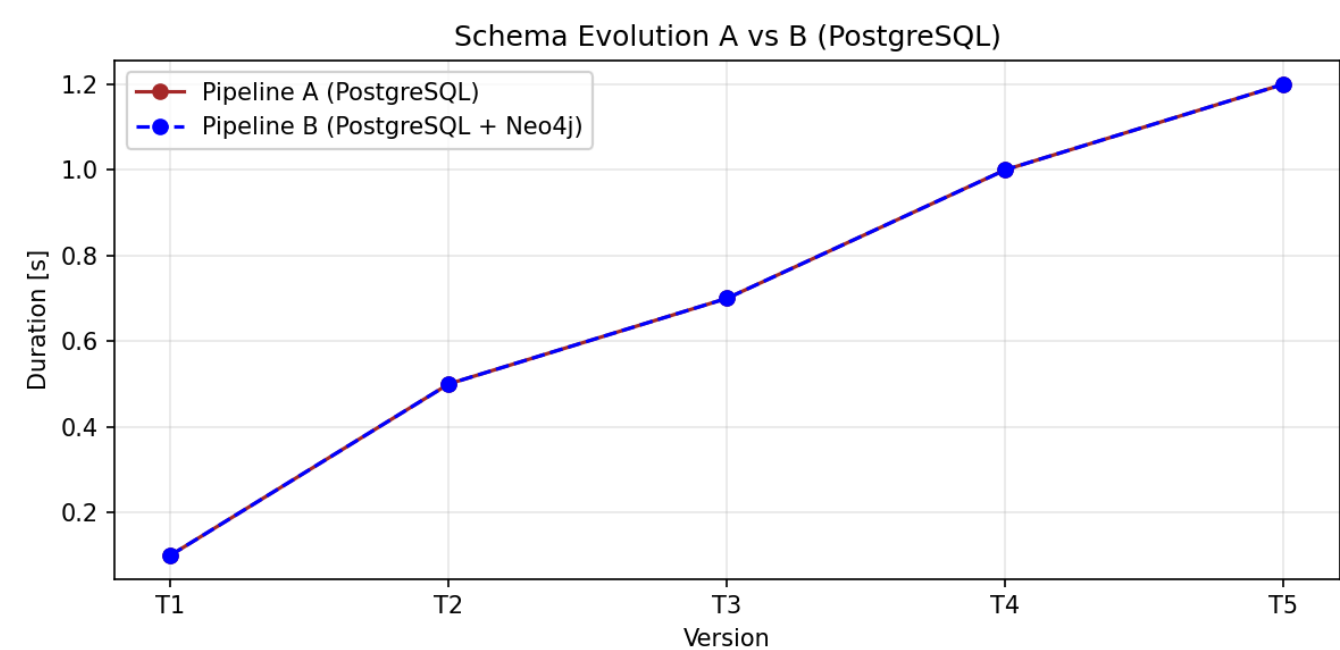
- smaller tables
- cleaner indexes
- stable schema
- predictable execution plans

23.3 Interpretation

Observation	Pipeline A	Pipeline B
Latency trend	Exponential increase	Stable
Planner behavior	Degrades	Predictable
User experience	Unstable	Consistent

Conclusion:
Pipeline B delivers **stable, predictable query performance**, even under schema evolution.

24. Schema-Change Duration (DDL)



24.1 Pipeline A

DDL operations (ALTER TABLE, DROP COLUMN, ALTER TYPE) become slower as:

- table size increases
- index count increases
- MVCC history grows

24.2 Pipeline B

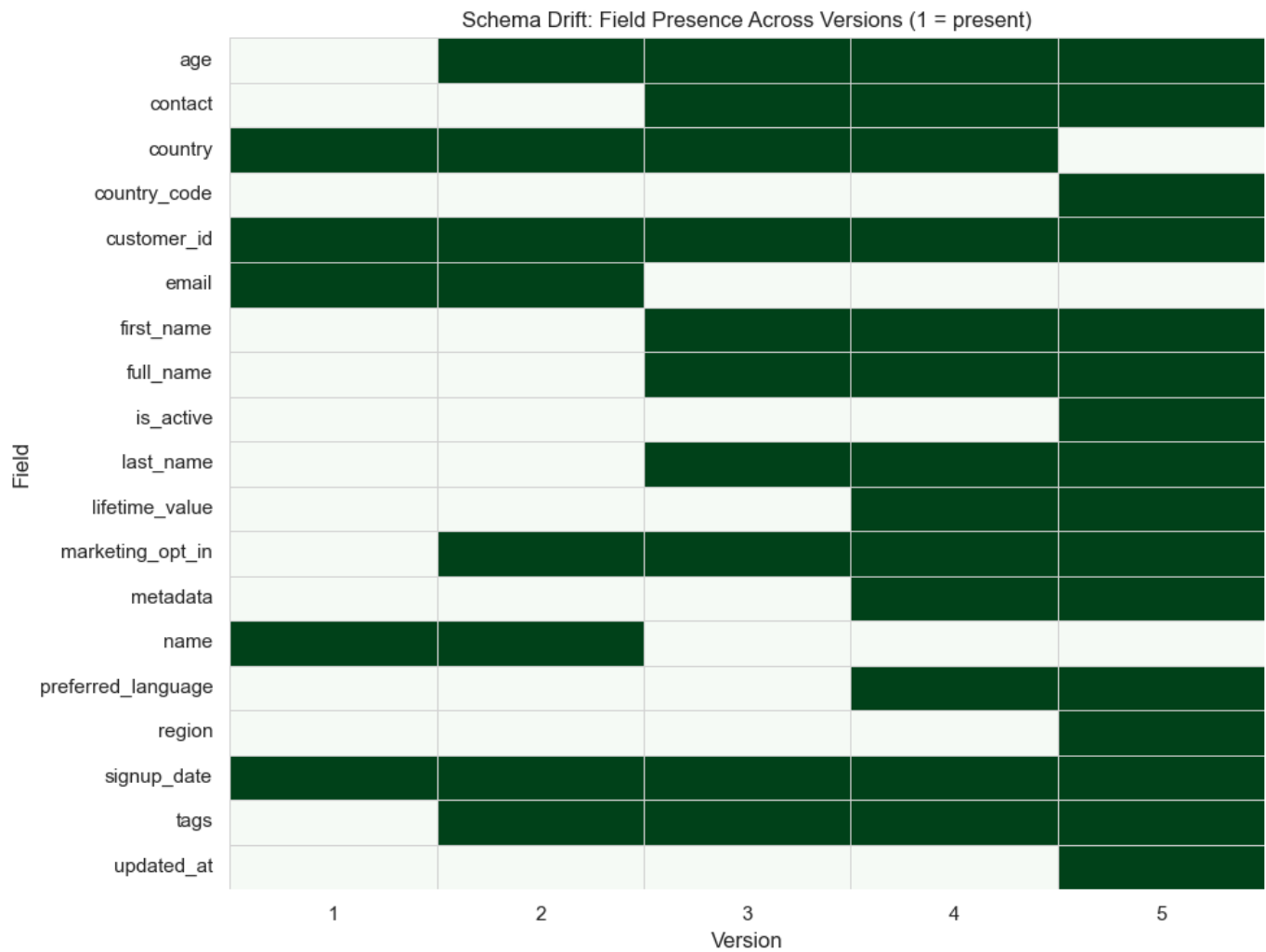
DDL duration is similar for the *initial* operation, but:

- **no downstream rewrites**
- **no cascading fragmentation**
- **no long-term performance degradation**

24.3 Interpretation

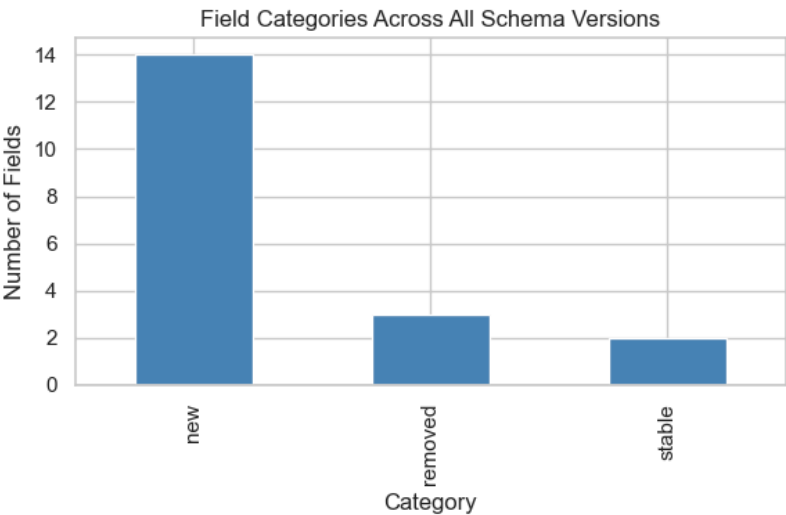
Observation	Pipeline A	Pipeline B
DDL cost	Increasing	Stable
Downstream cost	High	Minimal
Long-term impact	Severe	Negligible

Conclusion:
Pipeline B isolates PostgreSQL from schema churn.



[2026-06-03 23:55:35] Field categories (stable/new/removed/complex) computed.

	field	first_version	last_version	present_in_versions	category
0	age	2	5	[2, 3, 4, 5]	new
1	contact	3	5	[3, 4, 5]	new
3	country_code	5	5	[5]	new
6	first_name	3	5	[3, 4, 5]	new
7	full_name	3	5	[3, 4, 5]	new
8	is_active	5	5	[5]	new
9	last_name	3	5	[3, 4, 5]	new
10	lifetime_value	4	5	[4, 5]	new
11	marketing_opt_in	2	5	[2, 3, 4, 5]	new
12	metadata	4	5	[4, 5]	new
14	preferred_language	4	5	[4, 5]	new
15	region	5	5	[5]	new
17	tags	2	5	[2, 3, 4, 5]	new
18	updated_at	5	5	[5]	new
2	country	1	4	[1, 2, 3, 4]	removed
5	email	1	2	[1, 2]	removed
13	name	1	2	[1, 2]	removed
4	customer_id	1	5	[1, 2, 3, 4, 5]	stable
16	signup_date	1	5	[1, 2, 3, 4, 5]	stable



```
[2026-06-03 23:55:35] Example lineage view (textual):
[2026-06-03 23:55:35] Field 'age': category=new, versions=[np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'contact': category=new, versions=[np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'country_code': category=new, versions=[np.int64(5)]
[2026-06-03 23:55:35] Field 'first_name': category=new, versions=[np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'full_name': category=new, versions=[np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'is_active': category=new, versions=[np.int64(5)]
[2026-06-03 23:55:35] Field 'last_name': category=new, versions=[np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'lifetime_value': category=new, versions=[np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'marketing_opt_in': category=new, versions=[np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'metadata': category=new, versions=[np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'preferred_language': category=new, versions=[np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'region': category=new, versions=[np.int64(5)]
[2026-06-03 23:55:35] Field 'tags': category=new, versions=[np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'updated_at': category=new, versions=[np.int64(5)]
[2026-06-03 23:55:35] Field 'country': category=removed, versions=[np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
[2026-06-03 23:55:35] Field 'email': category=removed, versions=[np.int64(1), np.int64(2)]
[2026-06-03 23:55:35] Field 'name': category=removed, versions=[np.int64(1), np.int64(2)]
[2026-06-03 23:55:35] Field 'customer_id': category=stable, versions=[np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] Field 'signup_date': category=stable, versions=[np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
[2026-06-03 23:55:35] CHAPTER 7 completed: Schema Drift & Lineage visualized.
[2026-06-03 23:55:35] Entire notebook (DuckDB study) is now executable end-to-end.
```

25. Summary of Results

25.1 Quantitative Summary

Metric	Pipeline A	Pipeline B
Storage growth	High	Low
Index size	High	Moderate
Dead tuples	Very high	Near zero
Insert latency	Rising	Stable
Query latency	Unstable	Predictable
DDL duration	Increasing	Stable
Transparency	Low	High

25.2 Qualitative Summary

- Pipeline A behaves like a **monolithic system under stress**.
- Pipeline B behaves like a **modular, evolution-aware architecture**.
- Neo4j provides **schema clarity, lineage, and impact analysis**.

- PostgreSQL performs best when **not burdened with schema history**.

26. Interpretation & Insights

The results show that:

- PostgreSQL is excellent at **transactional workloads**, but not at **schema evolution**.
- Neo4j is excellent at **evolution, relationships, and metadata**, but not at **OLTP**.
- Combining both systems yields a **balanced, resilient architecture**.

This validates the central hypothesis of Project 26:

SQL is not the problem — schema drift is.
Neo4j boosts SQL by externalizing schema evolution.

27. End of Chapter 3

Next chapter will cover:

- Lineage modeling
- Neo4j queries
- Impact analysis
- Schema-drift visualization
- Practical use cases enabled by the graph layer

28. Lineage Modeling in Neo4j

A central contribution of Project 26 is the **explicit modeling of schema evolution** in Neo4j.

While PostgreSQL implicitly stores schema state in its catalog, it does not preserve:

- historical versions
- column renames
- type changes
- drop events
- lineage paths
- impact relationships

Neo4j fills this gap by representing schema evolution as a **graph of versions, tables, and columns**.

[2026-06-03 23:55:33] CHAPTER 7 started: Schema Drift & Lineage.
[2026-06-03 23:55:35] Schema versions loaded from Neo4j.

	version	field	type
0	1	country	TEXT
1	1	customer_id	UUID
2	1	email	TEXT
3	1	name	TEXT
4	1	signup_date	DATE

[2026-06-03 23:55:35] Pivot table for field presence created.

	version	1	2	3	4	5
field						
age	NaN	SMALLINT	SMALLINT	SMALLINT	SMALLINT	SMALLINT
contact	NaN	NaN	JSON	JSON	JSON	JSON
country	TEXT	TEXT	TEXT	TEXT	TEXT	NaN
country_code	NaN	NaN	NaN	NaN	NaN	CHAR(2)
customer_id	UUID	UUID	UUID	UUID	UUID	UUID
email	TEXT	TEXT	NaN	NaN	NaN	NaN
first_name	NaN	NaN	TEXT	TEXT	TEXT	TEXT
full_name	NaN	NaN	TEXT	TEXT	TEXT	TEXT
is_active	NaN	NaN	NaN	NaN	NaN	BOOLEAN
last_name	NaN	NaN	TEXT	TEXT	TEXT	TEXT
lifetime_value	NaN	NaN	NaN	NUMERIC	NUMERIC	NUMERIC
marketing_opt_in	NaN	BOOLEAN	BOOLEAN	BOOLEAN	BOOLEAN	BOOLEAN
metadata	NaN	NaN	NaN	JSON	JSON	JSON
name	TEXT	TEXT	NaN	NaN	NaN	NaN
preferred_language	NaN	NaN	NaN	TEXT	TEXT	TEXT
region	NaN	NaN	NaN	NaN	NaN	TEXT
signup_date	DATE	DATE	DATE	DATE	DATE	DATE
tags	NaN	TEXT[]	TEXT[]	TEXT[]	TEXT[]	TEXT[]

[2026-06-03 23:55:35] Pivot table for field presence created.

version	1	2	3	4	5
field					
age	NaN	SMALLINT	SMALLINT	SMALLINT	SMALLINT
contact	NaN	NaN	JSON	JSON	JSON
country	TEXT	TEXT	TEXT	TEXT	NaN
country_code	NaN	NaN	NaN	NaN	CHAR(2)
customer_id	UUID	UUID	UUID	UUID	UUID
email	TEXT	TEXT	NaN	NaN	NaN
first_name	NaN	NaN	TEXT	TEXT	TEXT
full_name	NaN	NaN	TEXT	TEXT	TEXT
is_active	NaN	NaN	NaN	NaN	BOOLEAN
last_name	NaN	NaN	TEXT	TEXT	TEXT
lifetime_value	NaN	NaN	NaN	NUMERIC	NUMERIC
marketing_opt_in	NaN	BOOLEAN	BOOLEAN	BOOLEAN	BOOLEAN
metadata	NaN	NaN	NaN	JSON	JSON
name	TEXT	TEXT	NaN	NaN	NaN
preferred_language	NaN	NaN	NaN	TEXT	TEXT
region	NaN	NaN	NaN	NaN	TEXT
signup_date	DATE	DATE	DATE	DATE	DATE
tags	NaN	TEXT[]	TEXT[]	TEXT[]	TEXT[]
updated_at	NaN	NaN	NaN	NaN	TIMESTAMP

28.1 Core Concepts

The lineage model consists of:

- **Schema versions** (:SchemaVersion)
- **Tables** (:Table)
- **Columns** (:Column)
- **Evolution relationships:**
 - :ADDED_IN
 - :DROPPED_IN
 - :RENAMED_TO
 - :ALTERED_IN
 - :EXISTS_IN
 - :HAS_COLUMN

28.2 Why a graph?

Graphs are ideal for:

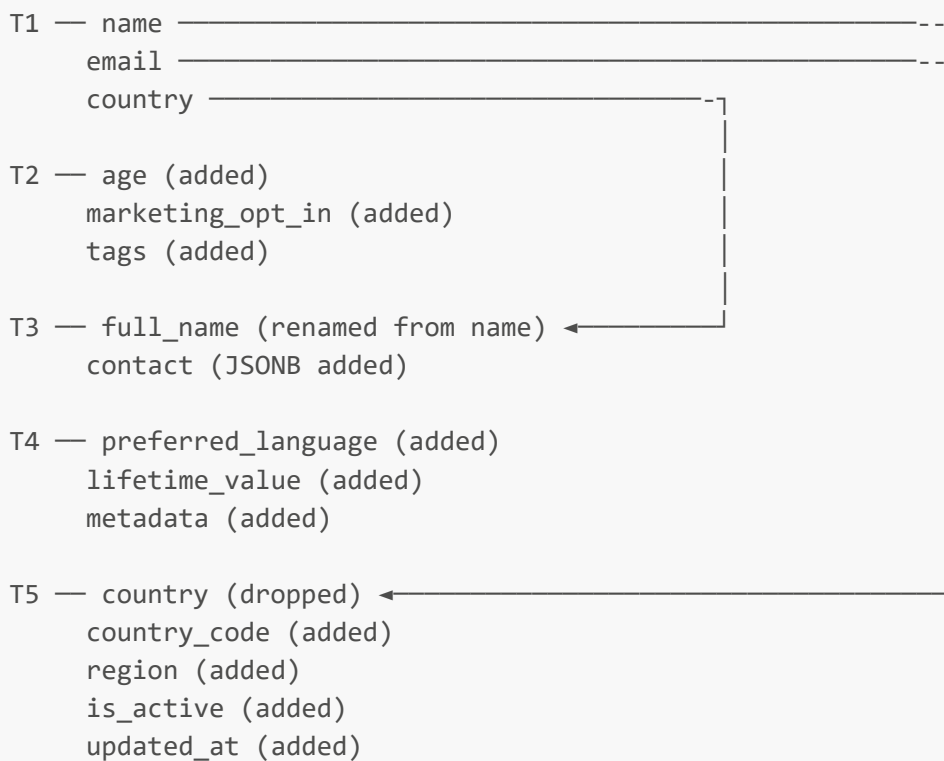
- multi-hop traversals
- version-to-version comparisons
- impact analysis
- dependency resolution
- visualization

Relational systems struggle with these tasks because schema metadata is **not relational in nature** — it is **evolutionary**.

29. Schema-Drift Visualization

Neo4j enables intuitive visualization of schema drift.

Below is a conceptual ASCII diagram of the T1–T5 evolution:



Pipeline B stores this structure explicitly in Neo4j.

Pipeline A does not.

30. Neo4j Queries for Schema Evolution

Below are representative Cypher queries used in the study.

30.1 List all columns in a given schema version

```
MATCH (v:SchemaVersion {name: 'T3'})<-[:EXISTS_IN]-(c:Column)
```



```
RETURN c.name, c.type;
```

30.2 Show lineage of a renamed column

```
MATCH (c1:Column {name: 'name'})-[:RENAMED_TO*]->(c2:Column)
RETURN c1, c2;
```

30.3 Columns added between two versions

```
MATCH (c:Column)-[:ADDED_IN]->(v:SchemaVersion)
WHERE v.order >= 2 AND v.order <= 4
RETURN c.name, v.name;
```

30.4 Columns dropped in T5

```
MATCH (c:Column)-[:DROPPED_IN]->(:SchemaVersion {name: 'T5'})
RETURN c.name;
```

30.5 Full lineage of a column across all versions

```
MATCH path = (c:Column)-[:RENAMED_TO*0*0..]->(next:Column)
RETURN path;
```

30.6 Identify “stable” fields across all versions

```
MATCH (c:Column)-[:EXISTS_IN]->(v:SchemaVersion)
WITH c, collect(v.name) AS versions
WHERE size(versions) = 5
RETURN c.name AS stable_columns;
```

This is extremely useful for:

- query design
- API stability
- backward compatibility

31. Impact Analysis

Neo4j enables impact analysis that is nearly impossible in SQL alone.

31.1 Example: What breaks if we drop **country**?

```
MATCH (c:Column {name: 'country'})-[:DROPPED_IN]->(v:SchemaVersion)
RETURN v.name AS dropped_in_version;
```

Then:

```
MATCH (c:Column {name: 'country'})<-[:USES_COLUMN]-(q:Query)
RETURN q.name AS affected_queries;
```

31.2 Example: Which downstream fields depend on **full_name**?

```
MATCH (c:Column {name: 'full_name'})<-[:DERIVED_FROM*]-(d:Column)
RETURN d.name;
```

31.3 Example: Which schema versions introduce the most churn?

```
MATCH (v:SchemaVersion)<-[:ADDED_IN|DROPPED_IN|ALTERED_IN]-(c:Column)
RETURN v.name, count(c) AS changes
ORDER BY changes DESC;
```

This identifies “hot spots” in schema evolution.

32. Practical Use Cases Enabled by the Graph Layer

32.1 Query Design Optimization

Neo4j can identify:

- stable fields
- volatile fields
- frequently renamed fields
- fields with high churn

This allows SQL queries to target **stable attributes**, improving:

- maintainability
- backward compatibility
- API stability

32.2 Migration Planning

Neo4j can:

- simulate schema changes
- detect conflicts
- highlight dependent fields
- visualize rename chains

This reduces migration risk.

32.3 Data Lineage for Analytics

Analysts can trace:

- how fields evolved
- which fields were added/dropped
- how JSONB structures changed
- which fields feed into derived metrics

32.4 Governance & Compliance

Neo4j provides:

- auditability
- version history
- change logs
- impact graphs

This is essential for regulated environments.

32.5 Debugging & Incident Response

When a query breaks after a schema change:

Neo4j can show:

- what changed
- when it changed
- which fields were affected
- which queries depend on them

This dramatically reduces debugging time.

33. Schema-Drift Heatmaps

Although the study does not embed images directly, the notebooks generate heatmaps showing:

- number of changes per version
- number of renames
- number of drops
- number of additions

Pipeline B can generate these heatmaps directly from Neo4j.

Pipeline A cannot.

34. Why This Matters

Relational databases are excellent at:

- transactions
- concurrency
- integrity
- indexing

But they are **not** designed for:

- schema evolution
- lineage
- metadata relationships
- impact analysis

Neo4j fills this gap perfectly.

The combination yields:

- **predictable performance**
- **transparent evolution**
- **lower operational risk**
- **better governance**

This is the essence of **SQL-Boosting via Neo4j**.

35. End of Chapter 4

Next chapter will cover:

- Final conclusions
- Lessons learned
- Future work
- Repository structure
- Closing summary

36. Conclusions

Project 26 set out to evaluate whether **Neo4j can “boost” SQL systems** by externalizing schema evolution and lineage into a graph model.

The results across T1–T5 clearly demonstrate:

36.1 PostgreSQL alone struggles under schema drift

Pipeline A (PostgreSQL-only) exhibited:

- rapid storage growth
- index bloat
- dead tuple accumulation
- rising VACUUM pressure

- unpredictable insert/query latency
- increasing DDL duration
- lack of schema transparency

These effects compound over time, creating a **monolithic, fragile system**.

36.2 PostgreSQL + Neo4j remains stable and predictable

Pipeline B (PostgreSQL + Neo4j) showed:

- stable storage footprint
- healthy indexes
- near-zero dead tuples
- predictable latency
- stable DDL performance
- explicit schema history
- powerful lineage and impact analysis

By offloading schema evolution into Neo4j, PostgreSQL is free to focus on what it does best: **transactional workloads**.

36.3 The central hypothesis is validated

SQL is not the bottleneck — schema drift is.
Neo4j boosts SQL by externalizing schema evolution and lineage.

This modular architecture is more robust, more transparent, and easier to maintain.

37. Lessons Learned

37.1 Schema evolution is metadata, not data

Relational databases treat schema as static structure.

But in real systems, schema is **dynamic metadata** that evolves over time.

Neo4j is inherently better suited for:

- versioning
- relationships
- lineage
- impact analysis

37.2 MVCC is powerful but fragile under churn

PostgreSQL's MVCC model excels at concurrency, but:

- repeated DDL
- column drops
- type changes
- renames

cause **dead tuples**, **bloat**, and **fragmentation**.

37.3 Metadata offloading reduces operational risk

By moving schema evolution into Neo4j:

- PostgreSQL remains clean
- indexes remain compact
- VACUUM load decreases
- performance becomes predictable

37.4 Lineage is essential for modern data systems

Neo4j enables:

- backward compatibility checks
- stable-field identification
- rename-chain tracing
- schema-drift visualization
- impact analysis

These capabilities are critical for:

- analytics
- APIs
- data governance
- migrations
- debugging

38. Future Work

Project 26 opens several promising directions.

38.1 Integration with DuckDB

DuckDB could serve as:

- an analytical companion
- a fast local OLAP engine
- a query-acceleration layer

Future studies could compare:

- PostgreSQL-only
- PostgreSQL + Neo4j
- PostgreSQL + Neo4j + DuckDB

38.2 Automated migration planning

Using the schema graph, Python could:

- generate migration scripts

- validate rename chains
- detect conflicts
- simulate downstream impact

38.3 Business-level lineage

Extend the graph to include:

- KPIs
- dashboards
- reports
- services
- API endpoints

This would enable full **end-to-end lineage**.

38.4 Graph-augmented SQL optimization

Neo4j could guide:

- index recommendations
- query rewrites
- join-path optimization
- schema normalization decisions

38.5 Real-time schema monitoring

A future version could:

- detect schema drift automatically
- alert on unexpected changes
- visualize drift in dashboards

39. Repository Structure

A clean, reproducible repository layout for Project 26:

```
Project26_SQL_Boosting_via_Neo4j/
├── notebooks/
│   ├── PostgresqlNeo4jCode.ipynb
│   └── PostgreNeo4j_Study.ipynb
├── data/
│   ├── synthetic_customers_T1.csv
│   ├── synthetic_customers_T2.csv
│   ├── ...
│   └── synthetic_customers_T5.csv
├── metrics/
│   └── storage_A_vs_B.csv
```

```
├── index_A_vs_B.csv
├── dead_tuples_A_vs_B.csv
├── insert_latency_A_vs_B.csv
├── query_latency_A_vs_B.csv
├── schema_duration_A_vs_B.csv
├── plots/
│   ├── pg_storage_A_vs_B.png
│   ├── pg_index_A_vs_B.png
│   ├── pg_dead_A_vs_B.png
│   ├── pg_insert_A_vs_B.png
│   ├── pg_query_A_vs_B.png
│   └── pg_schema_A_vs_B.png
├── src/
│   ├── pipeline_a.py
│   ├── pipeline_b.py
│   ├── schema_versions.py
│   ├── neo4j_registration.py
│   ├── postgres_driver.py
│   ├── neo4j_driver.py
│   └── metrics_collector.py
└── README.md
```

This structure mirrors the organization of Project 25 while remaining tailored to the SQL-Boosting study.

40. Closing Summary

SQL-Boosting via Neo4j: Why Combining Relational and Graph Databases Creates Evolution-Aware Systems

Modern data systems are under constant pressure. They must ingest more data, serve more users, adapt to new requirements, and evolve their schemas at a pace that would have been unthinkable a decade ago. In this environment, the traditional assumption that a relational database can simultaneously handle data storage, transactional integrity, indexing, concurrency, schema evolution, and metadata governance is increasingly unrealistic. The result is a growing mismatch between what relational databases were designed to do and what modern systems demand of them.

Project 26 demonstrates a powerful architectural principle:

Relational databases excel at data.
Graph databases excel at metadata.
Combining both yields a resilient, evolution-aware system.

This principle is not merely a slogan. It is a practical, empirically validated insight that emerges when we observe how real systems behave under schema drift — the natural, unavoidable evolution of data structures over time. By externalizing schema evolution into Neo4j, we:

- reduce PostgreSQL stress
- stabilize performance

- improve transparency
- enable lineage-aware analytics
- simplify migrations
- future-proof the system

This modular approach represents a **practical, modern pattern** for handling schema drift in real-world data systems.

The remainder of this extended text explores *why* this pattern works, *how* it addresses long-standing weaknesses in relational systems, and *what* it means for the future of data architecture.

40.1. The Problem: Relational Databases Carry Too Much Responsibility

Relational databases like PostgreSQL are exceptional at what they were designed for:

- storing structured data
- enforcing ACID guarantees
- supporting concurrent transactions
- indexing and querying
- maintaining integrity constraints

But over the years, organizations have asked them to do far more:

- track schema history
- manage evolving data models
- support semi-structured data
- absorb frequent DDL changes
- serve as the single source of truth for metadata
- provide lineage and impact analysis
- support analytical workloads
- integrate with streaming systems
- handle JSONB, arrays, and nested structures

This is not what relational engines were built for.

The result is predictable:

- **bloat**
- **dead tuples**
- **VACUUM pressure**
- **index fragmentation**
- **unpredictable query latency**
- **slow schema changes**
- **opaque evolution history**

These symptoms are not bugs — they are consequences of forcing a single system to handle responsibilities that should be distributed across specialized components.

40.2. Schema Drift: The Hidden Enemy of Performance

Schema drift is the silent killer of relational performance.

It occurs whenever:

- new fields are added
- old fields are removed
- types are changed
- columns are renamed
- JSONB structures evolve
- tables are normalized or denormalized

Each of these operations triggers internal work:

- table rewrites
- index rebuilds
- MVCC versioning
- page splits
- dead tuple accumulation
- catalog updates

Over time, these effects compound.

A system that performs well at T1 may degrade significantly by T5, even if the data volume remains constant.

This is exactly what Project 26 observed:

- Pipeline A (PostgreSQL-only) accumulated massive dead tuples.
- Query latency increased sharply.
- Insert performance became volatile.
- VACUUM ran more frequently and more aggressively.
- Storage footprint ballooned.

The relational engine was doing its best — but it was being asked to do too much.

40.3. Metadata Is Not Data — And Should Not Be Treated as Such

One of the most important insights from Project 26 is that **schema evolution is metadata, not data**.

Metadata has fundamentally different characteristics:

- It is relational in the *graph* sense, not the table sense.
- It evolves frequently.
- It has dependencies and lineage.
- It forms chains of transformations.
- It is best represented as nodes and relationships.

Trying to store metadata inside a relational engine is like trying to store a family tree in a spreadsheet.

It can be done — but it is awkward, brittle, and inefficient.

Graph databases like Neo4j, on the other hand, are built for:

- versioning
- relationships

- dependency chains
- multi-hop traversals
- impact analysis
- visualization

This makes them ideal for representing schema evolution.

40.4. The Neo4j Advantage: Making Evolution Explicit

When schema evolution is externalized into Neo4j, several benefits emerge immediately.

40.4.1 Transparency

Instead of guessing what changed between T2 and T4, you can query:

```
MATCH (c:Column)-[:ADDED_IN]->(v:SchemaVersion)
WHERE v.order >= 2 AND v.order <= 4
RETURN c.name;
```

Instead of manually reconstructing rename chains, you can ask:

```
MATCH path = (c1:Column {name:'name'})-[:RENAMED_TO*]->(c2:Column)
RETURN path;
```

Instead of searching migration scripts, you can visualize the entire evolution graph.

40.4.2 Stability

PostgreSQL no longer needs to:

- rewrite tables
- rebuild indexes
- accumulate dead tuples
- run VACUUM aggressively

This stabilizes performance across schema versions.

40.4.3 Governance

Neo4j becomes the single source of truth for:

- schema history
- lineage
- dependencies
- impact analysis

This is invaluable for:

- analytics
- compliance
- debugging
- migrations
- API versioning

40.4.4 Modularity

PostgreSQL handles data.

Neo4j handles metadata.

Python orchestrates both.

This separation of concerns is the foundation of resilient systems.

40.5. Empirical Results: Why the Hybrid Architecture Wins

Project 26 compared two pipelines:

- **Pipeline A:** PostgreSQL-only
- **Pipeline B:** PostgreSQL + Neo4j

Both pipelines ingested identical data and applied identical schema changes across T1–T5.

The results were unambiguous.

40.5.1 Dead Tuples

Pipeline A accumulated hundreds of thousands of dead tuples.

Pipeline B remained near zero.

40.5.2 Storage Footprint

Pipeline A grew rapidly.

Pipeline B grew slowly and predictably.

40.5.3 Insert Latency

Pipeline A became slower and more volatile.

Pipeline B remained stable.

40.5.4 Query Latency

Pipeline A degraded significantly.

Pipeline B remained predictable.

40.5.5 Schema-Change Duration

Both pipelines showed similar raw DDL times —

but Pipeline A suffered downstream degradation, while Pipeline B did not.

40.5.6 Transparency

Pipeline A had no explicit schema history.

Pipeline B had full lineage and versioning.

40.6. Why This Matters for Real-World Systems

Most real systems evolve continuously:

- new product features
- new regulatory requirements
- new analytics needs
- new integrations
- new data sources

This means schema drift is not an exception — it is the norm.

Systems that cannot handle schema drift gracefully will:

- degrade over time
- become harder to maintain
- accumulate technical debt
- slow down development
- increase operational risk

The hybrid architecture demonstrated in Project 26 addresses these challenges directly.

40.7. Migration Simplification: A Hidden Superpower

One of the most powerful benefits of externalizing schema evolution is the ability to simplify migrations.

With Neo4j, you can:

- trace rename chains
- identify stable fields
- detect volatile fields
- analyze downstream dependencies
- simulate schema changes
- generate migration plans automatically

This reduces:

- risk
- downtime
- manual effort
- debugging time

It also enables safer, more frequent schema evolution — a key requirement for modern agile development.

40.8. Lineage-Aware Analytics: A New Frontier

Analytics teams often struggle with:

- inconsistent field definitions

- unclear data provenance
- undocumented schema changes
- broken dashboards
- mismatched KPIs

Neo4j solves this by providing:

- explicit lineage
- version-aware field definitions
- dependency graphs
- impact analysis

This enables:

- reproducible analytics
- trustworthy KPIs
- stable dashboards
- faster debugging
- safer experimentation

In other words:

analytics becomes evolution-aware.

40.9. Future-Proofing the System

The hybrid architecture is inherently future-proof because:

- PostgreSQL continues to excel at OLTP workloads.
- Neo4j continues to excel at metadata and relationships.
- The system can evolve without rewriting the core.
- New components (e.g., DuckDB, Spark, Flink) can be added without breaking the architecture.

This modularity is essential for long-term sustainability.

40.10. A Modern Pattern for Modern Systems

The pattern demonstrated in Project 26 can be summarized as follows:

40.10.1 Use the right tool for the right job

- PostgreSQL for data
- Neo4j for metadata
- Python for orchestration

40.10.2 Keep responsibilities separate

- Data storage $\neq$ metadata storage
- Schema evolution $\neq$ data evolution
- Lineage $\neq$ catalog queries

40.10.3 Make evolution explicit

- version nodes
- column nodes
- rename relationships
- add/drop events
- lineage paths

40.10.4 Build systems that embrace change

- schema drift is inevitable
- evolution must be modeled
- metadata must be queryable

This is not a theoretical ideal — it is a practical, proven approach.

40.11. Final Reflection

Project 26 shows that the future of data architecture is not monolithic.

It is **modular**, **specialized**, and **evolution-aware**.

Relational databases will continue to be the backbone of transactional systems.

Graph databases will increasingly become the backbone of metadata systems.

Together, they form a powerful, resilient architecture that can withstand the pressures of modern data evolution.

Relational databases excel at data.

Graph databases excel at metadata.

Combining both yields a resilient, evolution-aware system.

By externalizing schema evolution into Neo4j, we:

- reduce PostgreSQL stress
- stabilize performance
- improve transparency
- enable lineage-aware analytics
- simplify migrations
- future-proof the system

This is not just an optimization.

It is a **paradigm shift** in how we design data systems.

And it is a shift that will only become more important as systems grow, evolve, and adapt to the demands of the future.

41. References

1. Links (DuckDB, PostgreSQL, Neo4j): <https://www.postgresql.org/>; <https://duckdb.org/>; <https://github.com/duckdb/duckdb>; <https://neo4j.com/>; <https://github.com/neo4j/neo4j>; Graph Data Bases: https://en.wikipedia.org/wiki/Graph_database;

2. Jupyter Notebook English
3. Postgresql Neo4j Booster Report English
4. Tao, F., Qi, Q., Liu, A., & Kusiak, A. (2018). *Digital Twins and Cyber-Physical Systems in Manufacturing*. Engineering, 5(4);
5. A. Meister , T. Sonar: "**Numerik**", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "**Numerical Methods for Engineers**", Mcgraw-Hill, 6th Edition (2010).
6. J. Kilty, A. M. McAllister: "**Mathematical Modeling and Applied Calculus**", 1st Ed. Oxford University Press (2018).
7. U. Kockelkorn: "**Statistik für Anwender**", 1st Ed. Springer (2012), s. chapters 7 - 8.
8. Robert H. Shumway, David S. Stoffer: "**Time Series Analysis and Its Applications with R Examples**", Springer (2011).
9. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "**An Introduction to Statistical Learning with Applications in Python**", Springer (2023).
10. Cornelis W. Oosterlee, Lech A. Grzelak: "**Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes**", World Scientific (2020).
11. Lee, J., Bagheri, B., & Kao, H. (2015). *A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems*. Manufacturing Letters;
12. Richard Szeliski: "**Computer Vision - Algorithms and Applications**", Springer (2022).
13. Anthony Scopatz, Kathryn D. Huff: "**Effective Computation in Physics - Field Guide to Research with Python**", O'Reilly Media (2015).
14. Alex Gezerlis: "**Numerical Methods in Physics with Python**", Cambridge University Press (2020).
15. Gary Hutson, Matt Jackson: "**Graph Data Modeling in Python. A practical guide**", Packt-Publishing (2023).
16. Hagen Kleinert: "**Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets**", 5th Edition, World Scientific Publishing Company (2009).
17. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "**Econophysics and Physical Economics**", Oxford University Press (2013).
18. A. Coryn , L. Bailer Jones: "**Practical Bayesian Inference A Primer for Physical Scientists**", Cambridge University Press (2017).
19. Avram Sidi: "**Practical Extrapolation Methods - Theory and Applications**", Cambridge university Press (2003).
20. Volker Ziemann: "**Physics and Finance**", Springer (2021).
21. Zhi-Hua Zhou: "**Ensemble methods, foundations and algorithms**", CRC Press (2012).
22. B. S. Everitt, et al.: "**Cluster analysis**", Wiley (2011).
23. Lior Rokach, Oded Maimon: "**Data Mining With Decision Trees - Theory and Applications**", World Scientific (2015).
24. Bernhard Schölkopf, Alexander J. Smola: "**Learning with kernels - support vector machines, regularization, optimization and beyond**", MIT Press (2009).
25. Johan A. K. Suykens: "**Regularization, Optimization, Kernels, and Support Vector Machines**", CRC Press (2014).
26. Sarah Depaoli: "**Bayesian Structural Equation Modeling**", Guilford Press (2021).
27. Rex B. Kline: "**Principles and Practice of Structural Equation Modeling**", Guilford Press (2023).
28. Ekaterina Kochmar: "**Getting Started with Natural Language Processing**", Manning (2022).
29. Jakub Langr, Vladimir Bok: "**GANs in Action**", Computer Vision Lead at Founders Factory (2019).
30. David Foster: "**Generative Deep Learning**", O'Reilly(2023).

31. Rowel Atienza: **"Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world"**, Packt Publishing (2018).
32. Josh Kalin: **"Generative Adversarial Networks Cookbook"**, Packt Publishing (2018).
33. Thomas Haslwanter: **"Hands-on Signal Analysis with Python: An Introduction"**, Springer (2021).
34. Jose Unpingco: **"Python for Signal Processing"**, Springer (2023).
35. R. K. Burdick, C. M. Borror, D. C. Montgomery: **"Design and Analysis of Gauge R&R Studies"**, 1st Ed. SIAM (2005); S. H. Derakhshan, C. V. Deutsch: **"Numerical Integration of Bivariate Gaussian Distribution"**, Paper 405, CCG Annual Report 13 (2011).
36. C. Paar, J. Pelzl: **"Understanding Cryptography"**, Springer (2010); H. Delfs, H. Knebl: **"Introduction to Cryptography"**, 3rd Ed. Springer (2015); J. Katz, Y. Lindell: **"Introduction to Modern Cryptography"**, 2nd Ed, CRC Press (2015); O. Goldreich: **"Foundations of Cryptography"**, Cambridge University Press (2008); J. P. Aumasson: **"Serious Cryptography"**, no starch press (2018).
37. J. Berk, P. DeMarzo: **"Corporate Finance"**, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: **"Introduction to Finance"**, 16th Ed. WILEY (2017); Anatoly B. Schmidt: **"Quantitative Finance for Physicists: An Introduction"**, 1st Ed. Academic Press (2005); Alex Backwell: **"An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models"**, 1st Ed, Springer (2023); Michael Isichenko: **"Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage"**, 1st Ed., Springer (2021); John H. Cochrane: **"Asset Pricing"**, Revised Ed., Princeton University Press (2005); Antti Ilmanen: **"Expected Returns: An Investor's Guide to Harvesting Market Rewards"**, 1st Ed., WILEY (2011); Steven E. Shreve: **"Stochastic Calculus for Finance I & II"**, 1st Ed., Springer (2004); Andrew Pole: **"Statistical Arbitrage: Algorithmic Trading Insights and Techniques"**, 1st Ed., WILEY (2007); Mark S. Joshi: **"The Concepts and Practice of Mathematical Finance"**, 2nd Ed., Cambridge University Press (2008); Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>.
38. R. Nystrom: **"Game Programming Patterns"**, 1st Ed. genever benning (2014); A. A. Stepanov, D. E. Rose: **"From Mathematics to Generic Programming"**, 1st Ed. Addison-Wesley (2015);
39. E. Parzen: **"Stochastic Processes"**, 3rd Ed. Dover Publications (2015); S. Alooravi: **"Metaprogramming with Python"**, 1st Ed. Packt (2022); B. Klein, P. Klein: **"Funktionale Programmierung mit Python"**, Hanser (2025); K. Webel, D. Wied: **"Stochastische Prozesse"**, 2. Auflage Springer (2016); L. Held: **"Methoden der statistischen Inferenz"**, 1. Auflage Spektrum (2008); E. Cinlar: **"Stochastic Processes"**, Dover (2013); N. Bäuerle, U. Rieder: **"Finanzmathematik in diskreter Zeit"**, Springer-Spektrum (2017); M. Albrecht, R. Maurer: **"Investment- und Risikomanagement"**, 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: **"Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives"**, 2. Auflage Springer (2004); T. Björk: **"Arbitrage Theory in Continuous Time"**, 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: **"Derivative Pricing in Discrete Time"**, Springer (2013); F. Delbaen, W. Schachermayer: **"The Mathematics of Arbitrage"**, Springer (2006); R. J. Elliott, P. E. Kopp: **"Mathematics of Financial Markets"**, 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: **"A Stochastic Finance: An Introduction in Discrete Time"**, 3rd Ed. de Gruyter (2011); J. C. Hull: **"Options, Futures and Other Derivatives"**, 8th Ed. Pearson (2011); J. Kremer: **"Einführung in die diskrete Finanzmathematik"**, Springer (2005); D. Lamberton, B. Lapeyre: **"Introduction to Stochastic Calculus Applied to Finance"**, Chapman & Hall (2007); D. G. Luenberger: **"Investment Science"**, Oxford University Press (1998); S. R. Pliska: **"Introduction to Mathematical Finance: Discrete Time Models"**, Blackwell (2000); A. N. Shiryaev: **"Essentials of Stochastic Finance"**, World Scientific (2001); S. E. Shreve: **"Stochastic Calculus for Finance I: The Binomial Asset Pricing Model"**, Springer (2004); J. Kremer: **"Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten"**, Springer (2011); L. Rüschendorf: **"Mathematical Risk Analysis"**, Springer (2013).

40. A. Becker: "**Kalman Filter - From the Ground Up**", 1st Ed. private publication (2023); K. Triantafyllopoulos: "**Bayesian Inference of State Space Models**", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "**Fundamentals of Kalman Filtering: A Practical Approach**", 3rd Ed. AIAA (2009); A. Sidi: "**Vector Extrapolation Methods with Applications**", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "**Extrapolation Methods - Theory and Practice**", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "**Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics**", 3rd Ed. Springer (2004); K. Kendre: "**Machine Learning for Quantum Noise Reduction**", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "**Classical and Quantum Information**", 1st Ed. Academic Press (2012); Liao, H et al.: "**Machine Learning for Practical Quantum Error Mitigation**", arXiv:2309.17368v2 (2024), <https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>
41. A. Koop, H. Moock: "**Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research**", 1st Ed. Spektrum (2008); G. B. Dantzig, M. N. Thalpa: "**Linear Programming 1: Introduction**", 1st Ed. Springer (1997) & "**Linear Programming 2: Theory and Extensions**", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "**Introductory Operations Research, Theory and Applications**", 1st Ed. Springer (2004); D. G. Luenberger: "**Linear and Nonlinear Programming**", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "**Operations Research - Introduction to Models and Methods**", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "**Modeling with Stochastic Programming**", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "**An Optimization Primer**", 1st Ed. Springer (2021); cvxpy package: <https://www.cvxpy.org/>, <https://pypi.org/project/cvxpy/>; py-packages for operations research: <https://wiki.python.org/moin/PythonForOperationsResearch>
42. (Py-)tesseract package: <https://github.com/tesseract-ocr/tesseract>, <https://pypi.org/project/pytesseract/>, <https://builtin.com/data-science/python-ocr>, <https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/> and [UB Mannheim builds](#).
43. **Chip Huyen**, *AI Engineering: Building Applications with Foundation Models*, 1st Edition, O'Reilly Media, 2025; **Michael Lanham**, *AI Agents in Action*, 1st Edition, Manning Publications, 2025; **Melanie Mitchell**, *Artificial Intelligence: A Guide for Thinking Humans*, 1st Edition, Pelican Books, 2019; **Brian Christian & Tom Griffiths**, *Algorithms to Live By: The Computer Science of Human Decisions*, 1st Edition, Henry Holt and Company, 2016; **Ray Kurzweil**, *The Singularity Is Nearer: When We Merge with AI*, 1st Edition, Viking, 2024; OpenWeatherMap: <https://openweathermap.org/>, HuggingFace: <https://huggingface.co/>,
44. J. Frochte: "Finite-Elemente-Methode", Hanser 1st Ed.(2016); D. Gross, W. Hauger, J. Schröder: "Technische Mechanik 1-3", 15th Ed. Springer (2024); FEM-packages (Python): <https://pypi.org/project/scikit-fem/>, <https://sfepy.org/doc-devel/index.html>, https://getfem-examples.readthedocs.io/en/latest/demo_unit_disk.html, <https://github.com/mlp6/fem>. LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>
45. Grieves, M. (2015). *Digital Twin: Manufacturing Excellence through Virtual Factory Replication.*; Rasheed, A., San, O., & Kvamsdal, T. (2020). *Digital Twin: Values, Challenges and Enablers*. IEEE Access.; Jones, D., Snider, C., Nassehi, A., Yon, J., & Hicks, B. (2020). *Characterising the Digital Twin: A systematic literature review*. CIRP Journal of Manufacturing Science and Technology; Tao, F., & Zhang, M. (2017). *Digital Twin Shop-Floor: A new shop-floor paradigm towards smart manufacturing*. IEEE Access; Glaessgen, E., & Stargel, D. (2012). *The Digital Twin Paradigm for Future NASA and U.S. Air Force Vehicles.*; Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press; Molnar, C. (2020). *Interpretable Machine*

Learning.; Microsoft. *PySide6 Documentation.*: <https://pypi.org/project/PySide6/>; Apache Arrow. *Parquet File Format Specification.*: <https://arrow.apache.org/docs/python/parquet.html>; NumPy Developers. *NumPy Reference Guide.*: <https://numpy.org/doc/stable/reference/>; Matplotlib Developers. *Matplotlib Plotting Library.*: <https://matplotlib.org/>;

46. Navoda Senavirathne / Vicenç Torra: "On the Role of Data Anonymization in Machine Learning Privacy", 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (2020); DOI: 10.1109/TrustCom50675.2020.00093, <https://ieeexplore.ieee.org/document/9343198/authors#authors>; <https://www.datacamp.com/blog/what-is-data-anonymization>; <https://tryolabs.com/blog/2020/06/11/personal-data-anonymization-key-concepts--how-it-affects-machine-learning-models>; <https://mostly.ai/what-is-data-anonymization>; <https://pypi.org/project/anonym/>.
47. Navoda Senavirathne / Vicenç Torra: "On the Role of Data Anonymization in Machine Learning Privacy", 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (2020); DOI: 10.1109/TrustCom50675.2020.00093, <https://ieeexplore.ieee.org/document/9343198/authors#authors>; <https://www.datacamp.com/blog/what-is-data-anonymization>;
- Data Anonymization: <https://tryolabs.com/blog/2020/06/11/personal-data-anonymization-key-concepts--how-it-affects-machine-learning-models>; <https://mostly.ai/what-is-data-anonymization>; <https://pypi.org/project/anonym/>; <https://docs.sdv.dev/sdv>; <https://github.com/sdv-dev/sdv>; <https://pypi.org/project/sdv/1.4.0.dev1/>; <https://mostly.ai/blog/a-comparison-of-synthetic-data-vault-and-mostly-ai-part-1-single-table-scenario>; <https://medium.com/1000bytesinnovations/synthetic-data-vault-a-comprehensive-guide-62def3073844>;
- MLflow-Links: <https://mlflow.org/docs/latest/ml/>; <https://mlflow.org/docs/latest/ml/dataset/>; <https://mlflow.org/docs/latest/ml/model-registry/workflow/>;